

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas
(TADS)

Disciplina: Tópicos Especiais de Programação

DOCUMENTAÇÃO TÉCNICA DE SOFTWARE

Detecção de Falhas em Impressão 3D

Sistema de Deep Learning para identificação automática
do Efeito Espaguete em impressoras FDM

Equipe: Amanda Iasmin · Izaque Nicolas · José Henrique · Matheus Ribeiro · Victor Sarrís · Sabrina Laís

Arquitetura: EfficientNetB3 · Transfer Learning · Django + React

Documento: Documentação completa do software

Data: Junho de 2026

Sumário

1. Visão Geral do Sistema
2. Definição do Problema
3. Objetivo da Solução
4. Base de Dados
5. Modelagem em Deep Learning
6. Arquitetura do Software
7. Avaliação dos Resultados
8. Aplicabilidade Real
9. Tecnologias Utilizadas
10. Estrutura do Repositório
11. Instalação e Execução
12. Limitações e Trabalhos Futuros
13. Conclusão

1. Visão Geral do Sistema

Este documento descreve, de forma completa, o software **Deteção de Falhas em Impressão 3D — Efeito Espaguete**, desenvolvido como atividade prática da disciplina de Tópicos Especiais de Programação do curso de Análise e Desenvolvimento de Sistemas. Trata-se de uma aplicação de *Deep Learning* aplicada à visão computacional, cujo propósito é classificar automaticamente imagens capturadas durante uma impressão 3D, distinguindo impressões saudáveis de impressões com falha.

O sistema é composto por três grandes blocos integrados: (1) um **modelo de rede neural convolucional** treinado por *Transfer Learning* sobre a arquitetura EfficientNetB3; (2) um **backend** em Django que expõe o modelo por meio de uma API REST de inferência; e (3) um **frontend** em React que permite ao usuário enviar uma foto da mesa de impressão e receber o diagnóstico com o respectivo grau de confiança. O resultado prático é uma ferramenta capaz de monitorar impressões longas sem supervisão humana contínua, emitindo um alerta de interrupção sempre que uma falha é detectada.

Em uma frase: o software recebe uma imagem da impressão em andamento e responde se a peça está sendo impressa corretamente (*success*) ou se já apresenta o **Efeito Espaguete** (*failure*), recomendando abortar a impressão em caso de falha.

2. Definição do Problema

2.1 Contexto

A impressão 3D por deposição de material fundido (FDM — *Fused Deposition Modeling*) é amplamente utilizada em prototipagem, manufatura distribuída e produção acadêmica. O processo, contudo, é suscetível a falhas silenciosas — em especial o chamado **Efeito Espaguete**, no qual o filamento passa a ser extrudado no ar, sem aderir ao modelo, resultando em uma malha caótica de plástico que inutiliza completamente a peça. Como impressões podem durar horas ou dias, uma falha que ocorre no início e não é percebida desperdiça todo o tempo e o material restantes.

2.2 Impacto

Falhas desse tipo geram consequências concretas:

- **Desperdício de filamento**, que pode custar dezenas de reais por carretel;
- **Perda de tempo** em impressões de longa duração que continuam mesmo já comprometidas;
- **Danos potenciais ao equipamento**, caso o filamento se acumule e obstrua componentes mecânicos;
- **Impacto financeiro** ampliado em contextos de produção em série.

2.3 Quem é afetado

São afetados *makers*, engenheiros, laboratórios de pesquisa, empresas de prototipagem e qualquer usuário de impressoras FDM de mesa aberta — especialmente aqueles que executam impressões longas sem acompanhamento presencial.

2.4 Relevância

A detecção automática de falhas elimina a necessidade de monitoramento humano constante, permite a interrupção imediata da impressão e reduz significativamente perdas de material e de tempo. Em ambientes com várias impressoras operando simultaneamente, o ganho de escala é ainda maior.

2.5 Justificativa do uso de Deep Learning

A detecção visual de padrões complexos e variáveis — como a morfologia irregular do Efeito Espaguete — é uma tarefa em que abordagens baseadas em regras explícitas tendem a falhar, pois não existe um formato único e previsível para a falha. Redes Neurais Convolucionais (CNNs) combinadas com *Transfer Learning* têm desempenho consolidado na literatura para classificação de imagens, inclusive com conjuntos de dados pequenos, o que torna essa abordagem a escolha mais adequada para o problema.

3. Objetivo da Solução

O sistema classifica automaticamente imagens capturadas durante a impressão 3D em duas categorias mutuamente exclusivas:

Classe	Descrição
success	Impressão ocorrendo normalmente
failure	Efeito Espaguete detectado

Resultado esperado: ao identificar uma falha, o sistema exibe um alerta e recomenda a interrupção imediata da impressão por meio do G-code `M112` (parada de emergência).

Ganho esperado: redução do desperdício de filamento e de tempo de máquina, com maior autonomia para o operador durante impressões longas.

4. Base de Dados

4.1 Origem

O conjunto de dados utilizado é o **3D Printing - SUCCESS - FAILURE DATASET (finetuned)**, disponível publicamente no Kaggle. Ele é derivado do projeto *open-source* **Obico / The Spaghetti**

Detective, que coleta imagens reais de impressoras FDM de mesa aberta ao redor do mundo, garantindo diversidade de modelos, ângulos e condições de iluminação.

4.2 Distribuição das amostras

O dataset é dividido em três conjuntos (treino, validação e teste), com balanceamento entre as classes:

Conjunto	failure	success	Total
Treino (train/)	114	114	228
Validação (valid/)	32	32	64
Teste (test/)	20	15	35
Total	166	161	327

4.3 Pré-processamento (ETL de imagens)

O carregamento e o tratamento das imagens seguem as seguintes etapas, implementadas no notebook de treino:

- **Redimensionamento:** todas as imagens são padronizadas para `300×300 pixels`, tamanho usado pelo modelo em produção. O notebook exploratório `DataPipeline.ipynb` chegou a usar `224×224` como teste inicial, mas o modelo final adota `300×300`;
- **Normalização:** os valores de pixel são tratados internamente pela própria EfficientNetB3, que já incorpora a etapa de normalização adequada aos pesos pré-treinados na ImageNet;
- **Carregamento eficiente:** uso de `image_dataset_from_directory` com `label_mode='binary'` e otimização de pipeline via `prefetch` com `tf.data.AUTOTUNE`;
- **Data Augmentation** (apenas no treino): inversão horizontal e vertical (`RandomFlip`), rotação de até 20% (`RandomRotation(0.2)`) e zoom de até 20% (`RandomZoom(0.2)`);
- **Balanceamento:** os conjuntos de treino e validação já estão balanceados (114 e 32 amostras por classe).

4.4 Limitações da base

- Dataset relativamente pequeno (327 imagens no total), o que pode limitar a generalização;
- Imagens capturadas em condições variadas de iluminação e ângulo de câmera;
- Possível viés para determinados modelos e marcas de impressoras;
- Cobre apenas o Efeito Espaguete como tipo de falha.

5. Modelagem em Deep Learning

O treinamento foi realizado em **duas fases**, aplicando *Transfer Learning* sobre a arquitetura **EfficientNetB3** pré-treinada na ImageNet. A cabeça de classificação foi customizada para o problema binário em questão.

5.1 Arquitetura da rede

Sobre a base convolucional da EfficientNetB3 (com `include_top=False`) foi acoplada uma cabeça de classificação composta por:

```
Input (300×300×3)
↓
Data Augmentation (RandomFlip · RandomRotation(0.2) · RandomZoom(0.2))
↓
EfficientNetB3 (base pré-treinada na ImageNet)
↓
GlobalAveragePooling2D
↓
Dropout (0.3)
↓
Dense (1, ativação sigmoid) → probabilidade de "success"
```

5.2 Fase 1 — Transfer Learning padrão

As camadas da base são **congeladas** (`base_model.trainable = False`) e apenas a cabeça de classificação é treinada. O objetivo é adaptar rapidamente o classificador ao novo domínio sem perturbar os pesos já aprendidos.

Parâmetro	Valor
Base	EfficientNetB3 (pesos ImageNet, <code>include_top=False</code>)
Camadas adicionadas	GlobalAveragePooling2D → Dropout(0.3) → Dense(1, sigmoid)
Otimizador	Adam (<code>lr = 0.001</code>)
Função de perda	Binary Crossentropy
Épocas	10
Batch size	32
Tamanho da imagem	300×300 px

5.3 Fase 2 — Fine-Tuning profundo

As **últimas 30 camadas** da base são descongeladas e treinadas com uma taxa de aprendizado bastante reduzida, para ajustar de forma fina os filtros convolucionais ao domínio específico das impressoras 3D sem destruir o conhecimento prévio.

Parâmetro	Valor
Camadas descongeladas	Últimas 30 da EfficientNetB3
Otimizador	Adam ($lr = 1e-5$)
Função de perda	Binary Crossentropy
Épocas	até 15 (com Early Stopping)
Callbacks	EarlyStopping(patience=4, monitor=val_loss, restore_best_weights) + ModelCheckpoint(monitor=val_accuracy, save_best_only)

O melhor modelo é salvo automaticamente no arquivo `melhor_modelo_efficientnet_pc.keras`, que é exatamente o artefato carregado pelo backend em produção.

5.4 Estratégias contra overfitting

- **Data Augmentation** (flip, rotação e zoom) integrado ao pipeline de treino;
- **Dropout de 30%** na camada densa;
- **Early Stopping** com restauração dos melhores pesos;
- **Congelamento progressivo** das camadas (base congelada na Fase 1, descongelamento parcial na Fase 2).

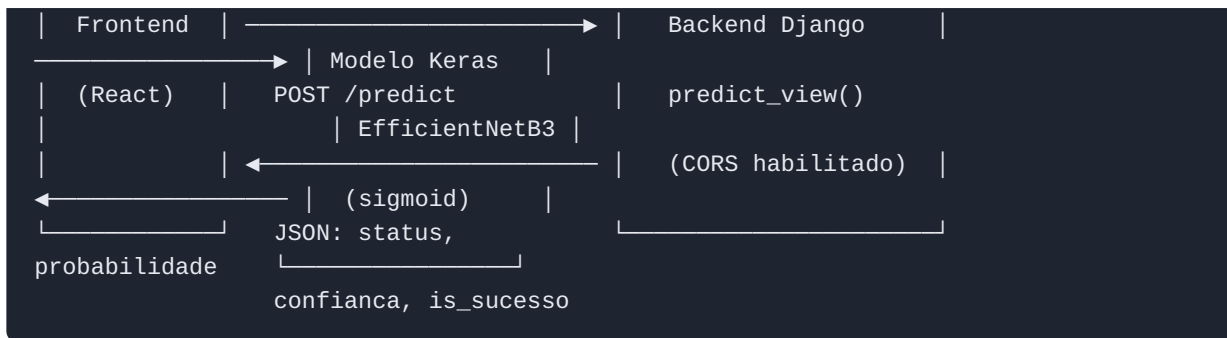
5.5 Justificativa da arquitetura

A EfficientNetB3 oferece um excelente equilíbrio entre acurácia e custo computacional, sendo adequada para inferência em ambientes com recursos limitados. Seu desempenho em tarefas de classificação de imagens com *fine-tuning* é consolidado na literatura, especialmente para datasets pequenos, o que a torna apropriada ao cenário deste projeto.

6. Arquitetura do Software

Além do modelo, o projeto entrega uma **aplicação web funcional completa**, dividida entre um backend de inferência e um frontend de interação. O diagrama abaixo resume o fluxo de dados ponta a ponta.





6.1 Backend — Django

O backend é uma aplicação Django (projeto `setup`, app `api`) cuja única responsabilidade é carregar o modelo treinado e expor um endpoint de inferência. O modelo é carregado **uma única vez**, no momento da inicialização do servidor, evitando o custo de recarregá-lo a cada requisição.

ENDPOINT DE PREDIÇÃO

Método	URL	Descrição
POST	<code>http://localhost:8000/predict</code>	Recebe uma imagem (<code>multipart/form-data</code> , campo <code>file</code>) e retorna o diagnóstico em JSON

A lógica de inferência, implementada em `api/views.py`, executa os seguintes passos:

1. Recebe o arquivo enviado no campo `file` (retorna erro 400 se ausente);
2. Abre a imagem com Pillow e a converte para RGB;
3. Redimensiona para `300×300` — exatamente o tamanho usado no treino;
4. Converte para um `array` NumPy e adiciona a dimensão de `batch`;
5. Executa `model.predict()` e obtém a probabilidade da classe `success`;
6. Aplica o limiar de 0,5: acima disso é `success`, abaixo é `failure`;
7. Calcula a confiança e devolve a resposta em JSON.

EXEMPLO DE RESPOSTA

```
{
  "status": "ALERTA: Falha Detectada (Efeito Espaguete)",
  "confianca": 94.37,
  "is_sucesso": false
}
```

Nota técnica: a confiança é calculada de forma assimétrica — quando o resultado é `success`, usa-se a própria probabilidade; quando é `failure`, usa-se o complemento (`1 - probabilidade`). Assim, a barra de confiança sempre reflete a certeza do modelo na classe que ele de fato escolheu. O

endpoint está com `@csrf_exempt` e o CORS liberado (`CORS_ALLOW_ALL_ORIGINS = True`) para permitir o consumo direto pelo frontend em desenvolvimento.

6.2 Frontend — React + Vite

A interface, construída em React 19 com Vite e estilizada com classes utilitárias (Tailwind), oferece uma experiência de uso simples e direta. O roteamento (`react-router-dom`) define duas rotas: a página inicial (`/`) e a página da ferramenta de análise (`/deep-learning`), apresentada como o *Monitor de Anomalias FDM*.

O componente principal de análise implementa o seguinte fluxo de interação:

1. **Upload da imagem** por clique ou *drag-and-drop*, com validação do tipo de arquivo (JPG, PNG, WEBP);
2. **Pré-visualização** da foto carregada antes do envio;
3. **Envio à API** via `fetch` com `FormData` para `POST /predict` ;
4. **Exibição do diagnóstico** com indicação visual (verde para impressão estável, vermelho para anomalia), o percentual de confiança e uma barra de progresso proporcional;
5. **Alerta de ação**: em caso de falha, a interface destaca a recomendação de abortar a impressão via G-code `M112` ;
6. **Tratamento de erros**, incluindo mensagem específica caso o backend Django não esteja em execução.

7. Avaliação dos Resultados

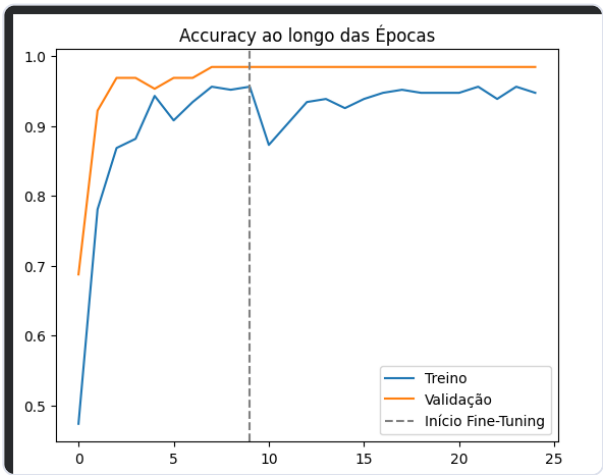
O modelo foi avaliado sobre o conjunto de teste (35 amostras nunca vistas durante o treino), utilizando um conjunto abrangente de métricas de classificação.

7.1 Métricas no conjunto de teste

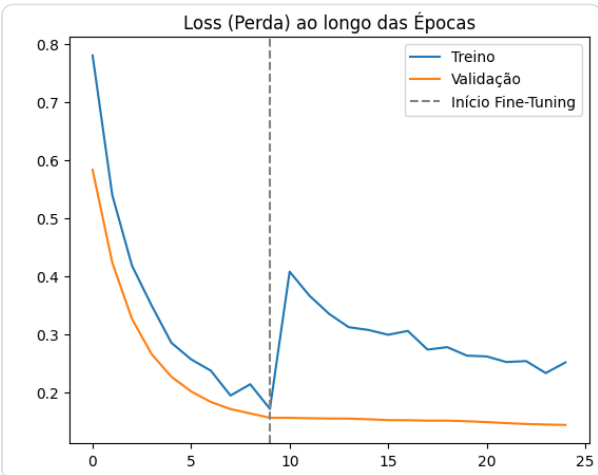
Métrica	Valor	Interpretação
Accuracy	85,71%	Proporção total de acertos
Precision	85,71%	Confiabilidade das previsões positivas
Recall	80,00%	Capacidade de encontrar os casos relevantes
F1-Score	82,76%	Equilíbrio entre precisão e revocação
ROC-AUC	0,967	Capacidade de separação entre as classes

7.2 Curvas de aprendizado

A acurácia de validação atingiu cerca de **98%** já nas primeiras épocas da Fase 1, mantendo-se estável durante o *fine-tuning*. A *loss* de validação convergiu para valores próximos a **0,15**, com comportamento estável. O pico de *loss* de treino no início da Fase 2 é esperado, decorrente do descongelamento das camadas e da redução drástica da taxa de aprendizado, normalizando rapidamente em seguida.



Acurácia de treino e validação ao longo das épocas

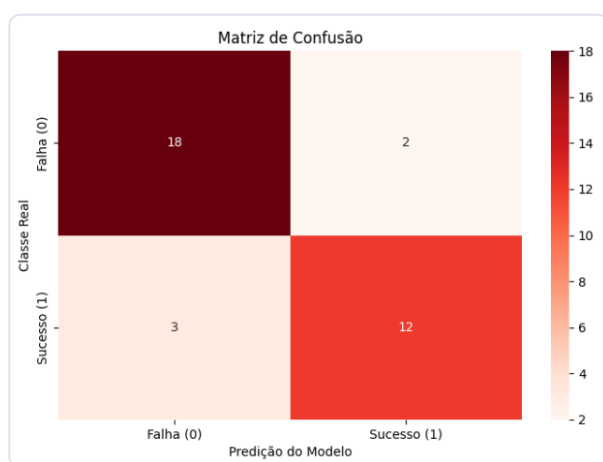


Loss de treino e validação ao longo das épocas

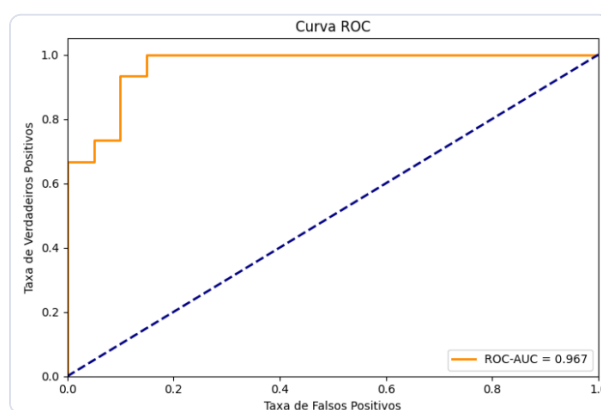
7.3 Matriz de confusão

	Predito: Falha	Predito: Sucesso
Real: Falha	18 ✓	2 ✗
Real: Sucesso	3 ✗	12 ✓

O modelo acertou **18 de 20** casos de falha (90% de detecção) e **12 de 15** casos de impressão saudável. Os 2 falsos negativos (falhas não detectadas) constituem o cenário mais crítico, pois permitiriam que a impressão continuasse com defeito. Os 3 falsos positivos (impressões saudáveis classificadas como falha) geram interrupções desnecessárias, porém são menos prejudiciais do ponto de vista de perda material.



Matriz de confusão no conjunto de teste



Curva ROC (AUC = 0,967)

7.4 Interpretação crítica

O ROC-AUC de **0,967** indica excelente capacidade de separação entre as classes. A acurácia de validação (~98%) é consideravelmente superior à acurácia de teste (85,71%), o que é esperado dado o tamanho reduzido do conjunto de teste (apenas 35 amostras), em que cada erro tem peso

proporcionalmente maior. Para um contexto de uso real, o ROC-AUC elevado é o indicador mais confiável da qualidade do modelo, pois mede a separabilidade das classes independentemente do limiar de decisão escolhido.

8. Aplicabilidade Real

O sistema pode ser integrado a qualquer impressora 3D FDM por meio de uma câmera de baixo custo conectada a um servidor local. A cada intervalo de tempo, uma foto da mesa de impressão é capturada, enviada ao backend Django e classificada pelo modelo. Em caso de falha detectada, a interface exibe um alerta com o diagnóstico e a recomendação de executar o G-code `M112` (interrupção de emergência).

Dimensão	Impacto
Econômico	Redução de desperdício de filamento e de tempo de máquina
Social	Democratiza a impressão 3D autônoma e confiável
Técnico	Integração possível com ecossistemas de automação existentes
Ambiental	Menos descarte de plástico por peças com falha

9. Tecnologias Utilizadas

Categoria	Tecnologia
Linguagem	Python 3.14
Deep Learning	TensorFlow / Keras
Arquitetura	EfficientNetB3 (Transfer Learning + Fine-Tuning)
Processamento de imagem	Pillow (PIL)
Análise e métricas	NumPy, Scikit-learn
Visualização	Matplotlib, Seaborn
Backend / API	Django 6 + django-cors-headers
Frontend	React 19 + Vite 8 + React Router
Notebooks	Jupyter Notebook

10. Estrutura do Repositório

```
Deep-Learning-special-topics-computing/
├── backend/                                # API de inferência (Django)
│   ├── api/
│   │   └── views.py                        # Endpoint POST /predict – carrega o modelo e
faz a predição
│   ├── setup/
│   │   ├── settings.py                    # Configurações Django (CORS habilitado)
│   │   └── urls.py                        # Roteamento: /predict → predict_view
│   ├── Deteccao_Falhas_Impressao3D.ipynb  # Notebook principal de treino
│   └── melhor_modelo_efficientnet_pc.keras # Modelo treinado (gerado pelo
notebook)
│   └── manage.py
├── frontend/                              # Interface web (React + Vite)
│   ├── src/
│   │   ├── App.jsx                        # Definição das rotas
│   │   ├── home.jsx                      # Página inicial
│   │   └── deepLearning.jsx              # Tela de upload, chamada à API e resultado
│   └── package.json                      # Dependências: React 19, Vite 8
├── notebooks/
│   ├── DataPipeline.ipynb                # Exploração e carregamento do dataset
│   └── Deteccao_Falhas_Impressao3D.ipynb # Cópia do notebook principal
├── data/                                  # Dataset de imagens (baixar via Kaggle)
│   ├── train/ (failure 114 · success 114)
│   ├── valid/ (failure 32 · success 32)
│   └── test/ (failure 20 · success 15)
├── .gitignore
└── README.md
```

11. Instalação e Execução

PRÉ-REQUISITOS

Python 3.14, Node.js 18+ e pip.

1. OBTER O DATASET

Baixar o dataset no Kaggle e extrair os arquivos nos diretórios `data/train/`, `data/valid/` e `data/test/`.

2. TREINAR O MODELO (NOTEBOOK)

```
pip install tensorflow matplotlib numpy scikit-learn seaborn jupyter pillow
cd backend
jupyter notebook Deteccao_Falhas_Impressao3D.ipynb
```

O notebook usa caminhos relativos ('train' , 'valid' , 'test') e deve ser executado a partir de backend/ . Como o dataset está na raiz em data/ , criam-se links simbólicos antes de rodar:

```
# Dentro de backend/
ln -s ../data/train train
ln -s ../data/valid valid
ln -s ../data/test test
```

Ao final, o arquivo melhor_modelo_efficientnet_pc.keras é salvo em backend/ .

3. INICIAR O BACKEND (DJANGO)

```
cd backend
pip install django django-cors-headers tensorflow pillow
python manage.py runserver
```

O servidor sobe em http://localhost:8000 e carrega o modelo automaticamente na inicialização.

4. INICIAR O FRONTEND (REACT)

```
cd frontend
npm install
npm run dev
```

Acessar http://localhost:5173 no navegador.

12. Limitações e Trabalhos Futuros

12.1 Limitações

- O modelo atua sobre **imagens estáticas**, sem análise temporal de quadros consecutivos;
- A câmera precisa de posicionamento e iluminação adequados para captura confiável;
- O dataset é relativamente pequeno (327 imagens), o que pode limitar a generalização;
- A solução detecta apenas o Efeito Espaguete, e não outros tipos de falha;
- Requer o backend Django em execução local para funcionar.

12.2 Trabalhos futuros

- Ampliar o dataset com maior diversidade de impressoras, filamentos e condições de iluminação;

- Implementar análise temporal (sequência de quadros) para detecção mais robusta;
- Suportar múltiplas classes de falha (*layer shifting*, *under-extrusion*, etc.);
- Converter o modelo para TensorFlow Lite, viabilizando inferência embarcada (Raspberry Pi, ESP32-CAM);
- Integrar diretamente com firmwares de impressora (Klipper, Marlin) para interrupção automática.

13. Conclusão

O software apresentado cumpre seu objetivo de detectar automaticamente o Efeito Espaguete em impressões 3D FDM, combinando um modelo de Deep Learning de bom desempenho (ROC-AUC de 0,967) a uma aplicação web funcional de ponta a ponta. A escolha do *Transfer Learning* com EfficientNetB3, somada às estratégias de regularização e ao balanceamento dos dados, permitiu obter resultados sólidos mesmo com um conjunto de dados reduzido.

A arquitetura desacoplada entre backend de inferência e frontend de interação torna o sistema simples de operar e extensível: novas classes de falha, análise temporal ou execução embarcada podem ser incorporadas sem reescrever a solução. Como ferramenta acadêmica, o projeto demonstra de forma concreta a aplicação de visão computacional a um problema real de manufatura, com impacto econômico, ambiental e operacional tangível.

Equipe de desenvolvimento: Amanda Iasmin · Izaque Nicolas · José Henrique · Matheus Ribeiro · Victor Sarrís · Sabrina Laís

Disciplina: Tópicos Especiais de Programação — Curso de Análise e Desenvolvimento de Sistemas (TADS)